ALPACA AI TRADING AGENTS HACKATHON · LABLAB.AI × ALPACA · 28 AUG – 4 SEP 2026

Sentinel

The model proposes. The gates decide.

An autonomous options trading agent whose language model cannot place an order — it can only propose one.

THE PROBLEM

An LLM trader's failure mode isn't being wrong.

It's being **confidently** wrong — at 3× the intended size, in a symbol it invented, using an order type nobody authorized. Most agent submissions this week will give the model a broker tool and let it call it. Sentinel does the opposite.

`The usual shape`	LLM holds broker credentials → LLM calls `place_order()` directly → whatever the model decided is now live.
`Sentinel's shape`	LLM emits a **structured proposal** — symbol, structure, stated max loss — with **no broker credentials and no submission tool**. Sixteen deterministic gates decide whether it becomes an order. The model cannot argue with them, route around them, or be prompted out of them.

ARCHITECTURE

Decision, verification, and execution are three different pieces of code.

`agent/proposer.py`	Claude receives ranked candidates from the quant engine, portfolio state and regime — selects ≤3, ranks them, writes the thesis. With no model available, the agent falls back to engine ranking. **A brain, never a finger.**
`gates/checks.py`	16 gates across Entry Authority, Data Readiness, Process Health, Release Integrity, Delivery Health. Every gate answers one question: *it is red — why should the agent not open new exposure right now?* Unregistered = BLOCKING, never a softer default.
`gates/safety_gate.py`	A durable, fail-closed permit: bound to the exact git commit and manifest SHA that produced it, expires in 90 minutes. Every failure path in this module returns refusal — there is no exception handler whose fallback is "allow".
`agent/executor.py`	Only accepts a proposal that (a) passed every pretrade gate and (b) matches an `order_shape` declared in the manifest. No shape declares `type: "market"` — a market order on a 15-min delayed feed is structurally impossible to emit.

AI LOGIC

A quant engine first. Four vectors consume its signal.

Regime (SPY/QQQ vs EMA20/50, breadth, RSI) → per-name score (trend, momentum, relative strength, RSI drift) → gap detector. Then:

VECTOR	BUDGET	WHAT IT TRADES	WHEN IT FIRES
Trend	45%	0–2 DTE debit verticals in the regime's direction, or credit verticals when IV is rich	score ≥ 0.55, RSI ≤ 65, not extended (measured, not assumed)
Catalyst	25%	ATM straddle on LULU (Sep 3 earnings) + post-earnings drift verticals on NVDA/CRM/CRWD (Aug 26)	confirmed, in-window calendar events only
Event	15%	1-DTE SPY strangle for the Aug jobs report; 0-DTE single-leg gap continuation on Sep 4	August Employment Situation, Sep 4 08:30 ET
Vol	15%	SPY iron condors, defined-risk, four legs	range-bound regime, elevated IV rank

RISK GATES — ENFORCED IN CODE, NONE ADVISORY

Every position is defined-risk. No naked shorts. No market orders.

4	2	1	1	2
Entry Authority	**Data Readiness**	**Delivery Health**	**Release Integrity**	**Process Health**
Is this account, this mode, allowed to trade?	*Is every input actually present?*	*If it breaks, does anyone find out?*	*Is running code the code that was verified?*	*Did the machinery obey its contract?*

$2,000

hard per-trade max loss (2% of $100k, fraction of starting equity — a drawdown shrinks absolute risk, never rescales it)

$13,000

portfolio at-risk cap across every open structure at once

–$3,000

daily kill switch: halts new entries, halves next day's sizing

$92,000

Entry Maintenance trip — new exposure stops, exits keep running

ALPACA INFRASTRUCTURE

Trading API, MCP server, and CLI — all three, deliberately.

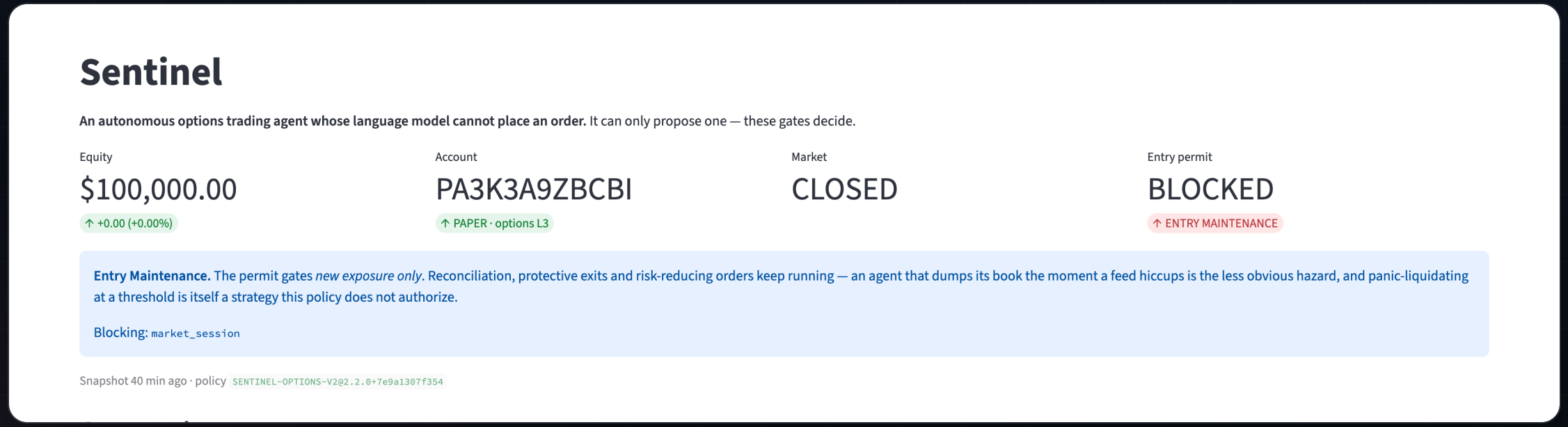
Trading API	Execution: limit-only, multi-leg via `LimitOrderRequest` with legs, DAY time-in-force. The programmable brokerage itself.
Market Data	IEX stocks in real time + indicative options snapshots delayed 15 minutes. Strikes are selected by snapshot delta, then **priced with Black-Scholes on the real-time underlying and snapshot IV** — delayed quotes select the contract, they never set the price.
MCP Server	`uvx alpaca-mcp-server`, wired via `.mcp.json` — the research/agent read path, and the theme of this hackathon.
Alpaca CLI	Ops and status: `alpaca account get`, `--dry-run` previews. Cron-friendly, structured JSON — matches the scheduled-cycle execution model.

Execution pipeline: `scripts/run_cycle.py` every 30 minutes through the entry window. The same cycle manages take-profit / stop-loss exits from ledger metadata — never leg by leg.

LIVE EVIDENCE — NOT JUST CLAIMS

The claim is checkable in the record, not asserted in this deck.

The agent writes a credential-free snapshot every cycle; the dashboard renders it. No Alpaca keys in the hosted page — judges verify "the model cannot place an order" by reading refused proposals, not by trusting the write-up.



RESULTS

Five trading days, judged live.

⏳ Filled in from the live account after the competition window closes

This deck is finalized before kickoff (2026-08-28). P&L, win rate by vector, and the equity curve are real numbers pulled from account **PA3K3A9ZBCBI** — reported honestly whichever way they land, not fabricated ahead of the event. Follow the live dashboard for the current state at any moment: **jralpha-sentinel.streamlit.app**

Six scheduled catalysts sit inside this exact window. The agent is **never directionally agnostic** on a catalyst day, never risks more than 2% of the account on one idea, and never holds an undefined-risk structure.

A catalyst-adaptive, defined-risk hedgehog is the only strategy that can be in the top decile of P&L *and* at the top of the risk-policy table at the same time.

Repo
github.com/JrWater/jralpha-sentinel · MIT

Live dashboard
jralpha-sentinel.streamlit.app

Account
PA3K3A9ZBCBI · Paper · $100,000